# CONCEPTION ET EVALUATION D'UNE APPLICATION MOBILE DE GESTION DE LIGNES DE COMMANDE (FLUTTER, API PHP, MYSQL)

## 1. Introduction et objectifs

Le travail répond à un sujet de TP : gérer les lignes d'une commande à partir d'un schéma relationnel à six tables (Client, Commande, ligneCommande, Produit, Region, Departement). Les fonctionnalités exigées sont : formulaire d'ajout d'un produit à une commande, affichage des produits, commandes et clients, modification d'un produit, suppression d'un produit dans une commande et tableau de bord.

## 2. Matériel et méthode

### 2.1 Architecture

L'application suit une architecture trois tiers. Flutter n'accède jamais directement à MySQL : il communique en HTTP/JSON avec une API PHP qui exécute les requêtes via PDO.

| Couche | Technologie | Rôle |
|---|---|---|
| Présentation | Flutter (Dart ≥ 3.3), package http | Interface adaptative (menu latéral ou barre de navigation) |
| Logique | PHP + PDO, 11 points d'accès | Validation, règles métier, agrégations |
| Données | MySQL (utf8mb4), XAMPP | Stockage relationnel |

### 2.2 Modèle de données

Le schéma comporte six tables. La table ligneCommande utilise une clé primaire composée (idCmd, idProd), qui interdit les doublons produit/commande, et une contrainte CHECK (qteCmd > 0). La suppression d'une commande entraîne celle de ses lignes (ON DELETE CASCADE). La région d'un client est mise à NULL si la région est supprimée (ON DELETE SET NULL).

### 2.3 Points d'accès de l'API

| Méthode | Point d'accès | Fonction |
|---|---|---|
| POST | login.php | Authentification |
| GET | clients.php, commandes.php,produits.php | Consultation |
| GET | lignes.php?idCmd= | Détail d'une commande |
| POST | produit_save.php | Ajout / modification d'un produit |
| POST | commande_add.php | Création d'une commande |
| POST | ligne_add.php | Ajout d'une ligne (avec contrôle du stock) |
| POST | ligne_delete.php | Suppression d'une ligne |
| GET | stats.php | Indicateurs du tableau de bord |

### 2.4 Mécanismes de fiabilité

- **Requêtes préparées** sur toutes les entrées, contre l'injection SQL.
- **Validation** par filter_var (entiers et flottants positifs).

- **Transaction avec SELECT … FOR UPDATE** dans ligne_add.php : le stock est lu sous verrou, comparé à la quantité cumulée (ligne existante + demande), puis la ligne est insérée ou mise à jour. Une quantité excédant le stock est refusée.
- **Agrégation côté SQL** (SUM, GROUP BY) pour limiter les transferts vers le client.
- **Gestion d'erreurs** homogène : réponse JSON {success, data | message} et codes HTTP 400, 401, 404, 500.

### 2.5 Interface

L'application compte 8 écrans/pages et 12 widgets réutilisables : connexion, tableau de bord, clients, commandes, détail d'une commande, produits. Les fonctions incluent la recherche, les étiquettes de niveau de stock, la confirmation avant suppression et la création de commandes.

## 3. Résultats

Le tableau de bord restitue en outre le chiffre d'affaires mensuel sur six mois (histogramme), les meilleurs clients et les produits à réapprovisionner.

### 3.3 Scénarios de validation

Huit scénarios sont définis : connexion valide et invalide, affichage du catalogue, ajout et modification d'un produit, ajout d'une ligne, refus pour stock insuffisant, mise à jour des statistiques, suppression d'une ligne. Ils sont décrits mais **non automatisés** : le projet ne contient aucun test unitaire ni d'intégration.

## 4. Discussion

### 4.1 Points forts

La séparation en trois couches protège les identifiants de la base. La combinaison clé composite, contrainte CHECK, requêtes préparées et transaction verrouillée garantit l'intégrité des lignes et évite les conflits d'accès concurrent lors de l'ajout. Le calcul statistique côté serveur est adapté à un volume croissant.

## Conclusion

L'application remplit l'ensemble des fonctionnalités demandées avec une architecture claire et des mécanismes d'intégrité solides pour un projet pédagogique local. Elle ne constitue toutefois pas un système exploitable en production. Les priorités d'évolution sont, dans l'ordre : (i) cohérence du stock, (ii) authentification et autorisation réelles, (iii) historisation des prix, (iv) HTTPS et durcissement, (v) tests automatisés et pagination.